

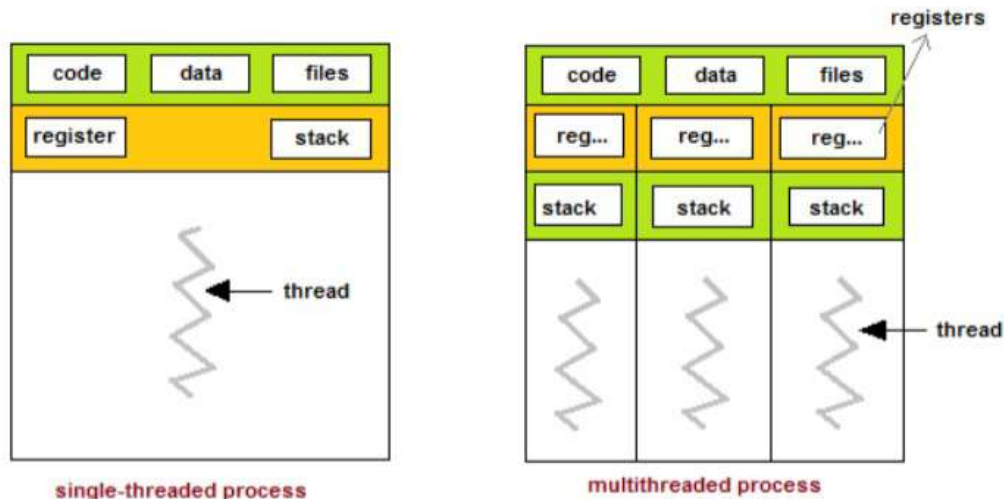
THREADS – OVERVIEW

A Thread is a basic unit of CPU utilization, consisting of a program counter, a stack, and a set of registers, (and a thread ID).

Traditional (heavyweight) processes have a single thread of control - There is one program counter, and one sequence of instructions that can be carried out at any given time.

A process can contain multiple threads.

A thread is also known as lightweight process.



Difference between Process and Thread

S.N.	Process	Thread
1	Process is heavy weight or resource intensive.	Thread is light weight, taking lesser resources than a process.
2	Process switching needs interaction with operating system.	Thread switching does not need to interact with operating system.
3	In multiple processing environments, each process executes the same code but has its own memory and file resources.	All threads can share same set of open files, child processes.
4	If one process is blocked, then no other process can execute until the first process is unblocked.	While one thread is blocked and waiting, a second thread in the same task can run.
5	Multiple processes without using threads use more resources.	Multiple threaded processes use fewer resources.
6	In multiple processes each process operates independently of the others.	One thread can read, write or change another thread's data.

Advantages of Thread

- Threads minimize the context switching time.
- Use of threads provides concurrency within a process.
- Efficient communication.
- It is more economical to create and context switch threads.
- Threads allow utilization of multiprocessor architectures to a greater scale and efficiency.

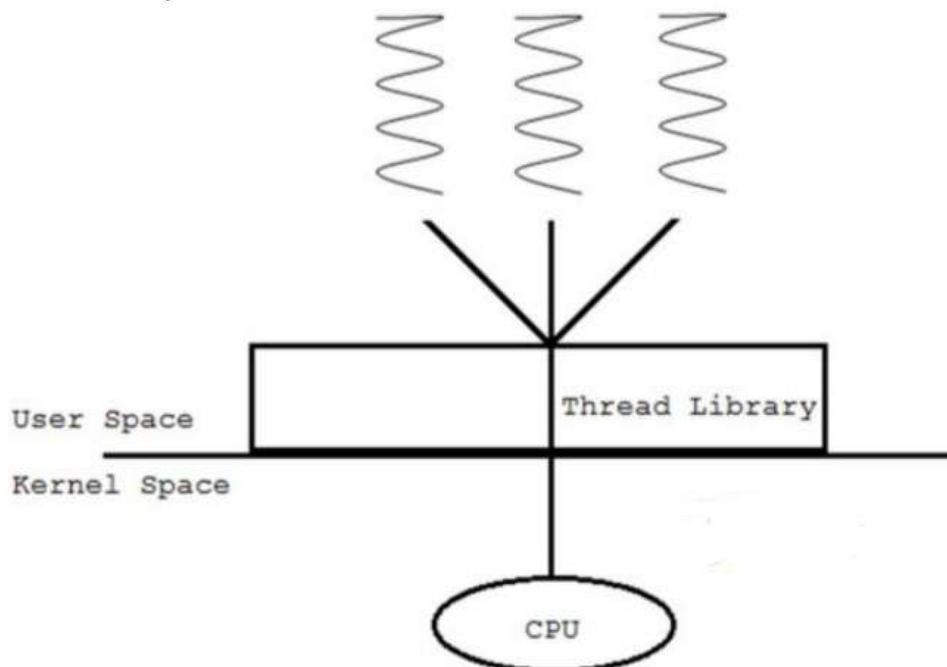
Types of Thread

Threads are implemented in following two ways:

- **User Level Threads** – User managed threads.
- **Kernel Level Threads** – Operating System managed threads acting on kernel, an operating system core.

User threads are above the kernel and without kernel support. These are the threads that application programmers use in their programs.

Kernel threads are supported within the kernel of the OS itself. All modern OSs support kernel level threads, allowing the kernel to perform multiple simultaneous tasks and/or to service multiple kernel system calls simultaneously.



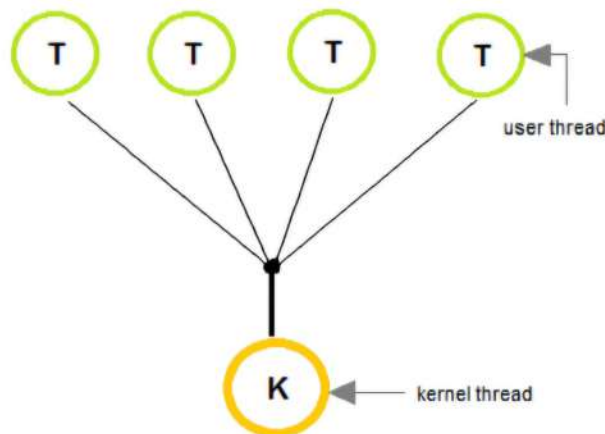
MULTITHREADING MODELS

The user threads must be mapped to kernel threads, by one of the following strategies:

- Many to One Model
- One to One Model
- Many to Many Model

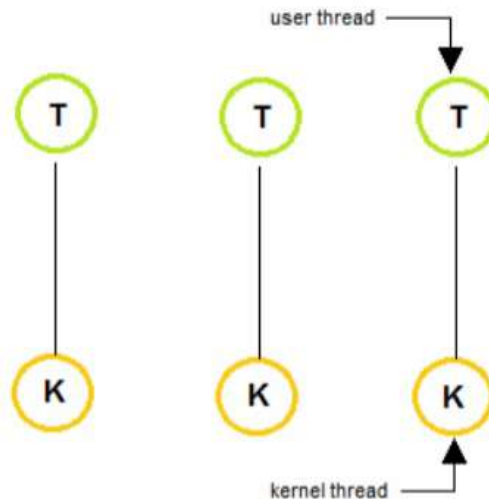
Many to One Model

In the many to one model, many user-level threads are all mapped onto a single kernel thread. Thread management is handled by the thread library in user space, which is efficient in nature.



One to One Model

The one to one model creates a separate kernel thread to handle each and every user thread. Most implementations of this model place a limit on how many threads can be created. Linux and Windows from 95 to XP implement the one-to-one model for threads.



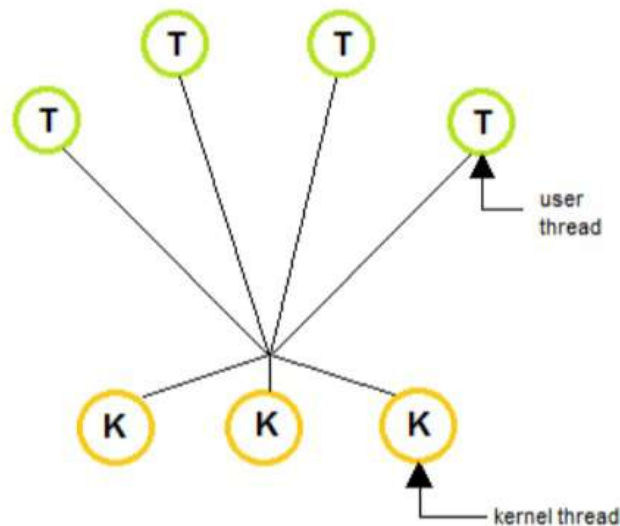
Many to Many Model

The many to many model multiplexes any number of user threads onto an equal or smaller number of kernel threads, combining the best features of the one-to-one and many-to-one models.

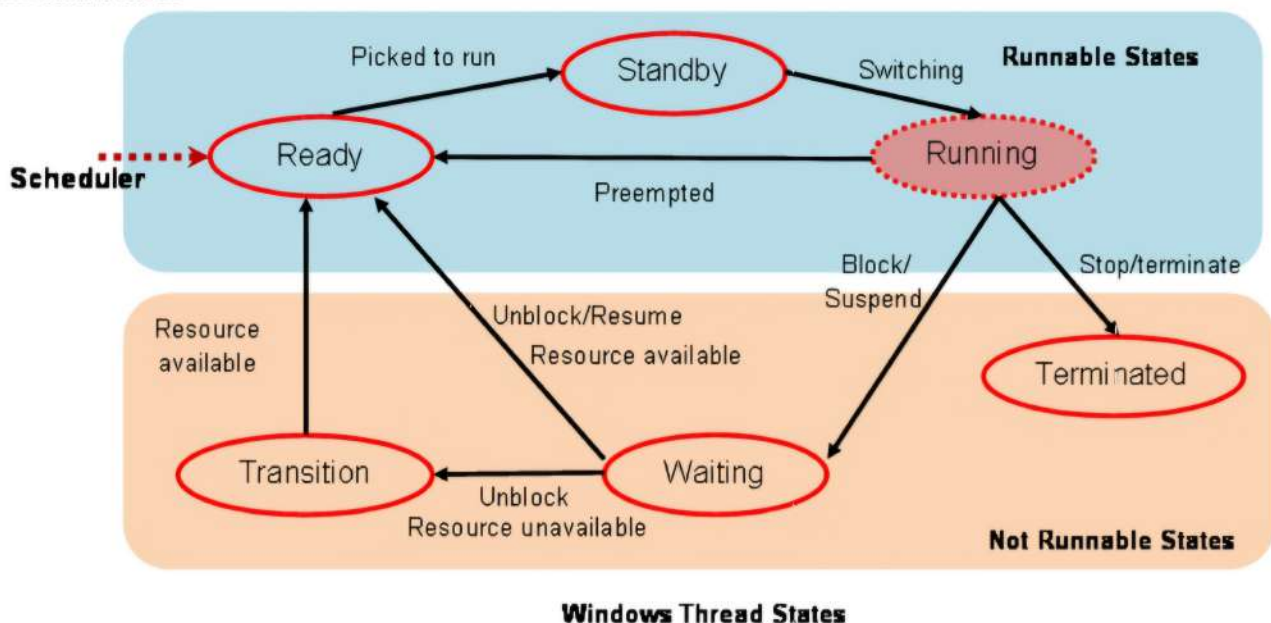
Users can create any number of the threads.

Blocking the kernel system calls does not block the entire process.

Processes can be split across multiple processors.



Thread States



A thread can be in the following states:

- **Ready** - it is prepared to run on the next available processor.
- **Running** - it is executing.
- **Standby** - it is about to run; only one thread may be in this state at a time.
- **Terminated** - it is finished executing.
- **Waiting** - it is not ready for the processor, when ready, it will be rescheduled.
- **Transition** - the thread is waiting for resources other than the processor,